DECISION PROPOSAL · TRANSCRIPT INSIGHT PIPELINE

Buy the intelligence, not the brand

A recommendation that the sermon transcript theme-extraction pipeline run through OpenRouter on a low-cost default model, with premium models reserved for adjudication and quality assurance — and the evidence, the arithmetic, and the objections that go with it.

Prepared for Gospel Light House leadership **Date** 27 August 2026

Decision required Credit approval & pilot go-ahead **Prices verified** 27 August 2026

CONTENTS

01 Executive summary
02 What the pipeline does today
03 The cost analysis
04 Why the price gap exists
05 Subscription, API, and our own asset
06 One key, many applications
07 Optionality and hybrid tiering
08 Risks and objections
09 The recommendation
10 Appendix: sources & assumptions

01 Executive summary

The request. Approve a **$100 prepaid OpenRouter balance** and a **paid pilot on 50 transcripts** (estimated cost: under $5). If the pilot's results hold up, adopt a low-cost model as the pipeline's production default, with a premium Claude model re-scoring the small subset of transcripts that actually reach publication.

Nothing has to be built. The pipeline is already provider-agnostic. Its configuration already names OpenRouter as the provider and a DeepSeek model as the default. Changing which model does the work is one line in a configuration file — not an engineering project, not a migration, not a cost. We are not asking permission to build the switch. We are asking permission to use the one that is already there.

The number. Analysing all 1,800 transcripts for one theme costs roughly **$154 on Claude Opus 5** and roughly **$1.89 on DeepSeek V4 Flash** — a factor of about **81×**. Against Claude Sonnet 5 the factor is about **33×**. These are modelled from the pipeline's own configuration constants and today's published prices; the calculator is in the repository and anyone can re-run it.

81× Cost ratio, Claude Opus 5 to DeepSeek V4 Flash, on one full-corpus run	**$1.89** One theme across all 1,800 transcripts on the proposed default
$7.02 Same run with Claude Sonnet 5 re-scoring the top 150 as quality assurance	**1 line** Configuration change required to switch models. No code change.

The argument is about iteration, not about a single invoice

$154 is not a frightening number, and this proposal will not pretend it is. Frontier model prices have fallen sharply — Claude Opus 5 costs a third of what the previous Opus generation did. One run of one theme at frontier prices is affordable.

But a run is never one run. **Every theme is a full pass over the corpus.** Ten themes is ten runs. Every prompt refinement, every schema correction, every re-analysis after new transcripts arrive is another full run. That is where the multiplier bites:

FIRST-YEAR RESEARCH PROGRAMME, 55-MINUTE AVERAGE TRANSCRIPT

SCENARIO	CLAUDE OPUS 5	CLAUDE SONNET 5	DEEPSEEK V4 FLASH
1 theme (baseline)	$153.88	$61.55	$1.89
10 themes	$1,538.82	$615.53	$18.90

SCENARIO	CLAUDE OPUS 5	CLAUDE SONNET 5	DEEPSEEK V4 FLASH
10 themes + 3 re-runs after prompt revisions	$2,000.47	$800.19	$24.57
Cost of discovering one bad prompt afterwards	$153.88	$61.55	$1.89

THE CORE OF THE CASE

An expensive model does not merely cost more. It makes the team afraid to run the pipeline. A budget that punishes iteration produces worse research, because themes that cannot be justified on an invoice simply never get explored, and a prompt nobody dares re-run never gets improved. **The cheap model does not buy us a discount. It buys us permission to be thorough.**

What we are not saying

This is not a proposal against Claude. Claude is excellent, and its chat interface is the best interactive tool of its kind — we are not proposing to cancel it, and the team should keep using it for exploratory and conversational work. Nor is it a claim that all models are equally capable. It is a narrower claim: *our* task is a bounded, well-specified one, and the current generation of mid-tier models does it reliably. We are paying frontier prices for a commodity operation.

BEFORE ANYTHING ELSE, ONE CORRECTION OF A COMMON ASSUMPTION

The Claude subscription we already pay for cannot fund this work — at any tier. A Claude subscription buys seats on Claude's chat apps; it does not include API access, and it cannot issue the key an application needs. In Anthropic's own words, it "doesn't include access to the Claude API or Console." These are two separate products on two separate accounts with two separate invoices. So the choice in front of us is not *subscription versus API* — that money is already committed elsewhere and cannot be redirected. The choice is which API to buy, and this document argues for the one that is not locked to a single vendor. Section 5 sets out the full distinction, including the one narrow exception and why it does not apply to us.

And OpenRouter is not a bet against Claude either. Claude Opus 5 is available through OpenRouter at exactly the same published price as buying it from Anthropic directly — $5 and $25 per million input and output tokens. Choosing OpenRouter costs us nothing in Claude access. It simply means we stop being locked into one vendor for work that does not require them.

The part of this decision with the longest reach

The cost argument is the easiest to verify, but it is not the largest thing on offer. An API key is not a line item for one project — it is **the ability to build our own software with AI as the engine**. The Claude interface we admire is itself just an application someone built on top of a model; with a key, we can build the version that fits Gospel Light House specifically. An assistant that answers from our own sermon archive and cites the sermon it drew from. Semantic search across everything ever preached. Automatic show notes, subtitles and translation for the media library. Drafting help inside the tools staff already use.

None of those require a new vendor, a new contract, or a new approval — they run on the same key and the same balance as the transcript pipeline. And because different jobs suit different engines, a router is what lets each one use the right model instead of forcing every job onto a single vendor's. Section 6 sets this out in full, with the catalogue and the limits.

> READ THIS BEFORE YOU READ ANY NUMBER BELOW
>
> **The corpus has not been measured yet.** Every figure in this document assumes 1,800 transcripts averaging 55 minutes. If the real average is 90 minutes, costs rise about 45%; if it is 150 minutes, transcripts cross the pipeline's chunking threshold and costs rise more than threefold. The cost calculator takes the true average as an input. **Measuring the corpus is the first task of the pilot**, and it is the single assumption most capable of moving these numbers.

---

02 What the pipeline does today

The Transcript Theme Extraction pipeline reads a corpus of sermon transcripts and, for a given theme, tells us which transcripts engage that theme, how directly, and exactly which passages carry it. Everything below is drawn from the code, not from recollection.

What happens to one transcript

1. **Length estimate.** The transcript's token count is estimated at roughly 3.5 characters per token.
2. **Under 20,000 tokens** — one API call analyses the whole transcript in a single pass.
3. **Over 20,000 tokens** — the transcript is split into 12,000-token chunks with 800 tokens of overlap. Each chunk is analysed in parallel, and one additional synthesis call merges the partial findings into a single judgement.
4. **The model returns strict JSON**: a relevance score from 0 to 100, an explicitness rating (explicit, tangential, or absent), a reasoning paragraph, and a list of passage locations marked by exact verbatim start and end phrases.
5. **Those markers are used to cut the real passages out of the source text**, which are assembled into a Word report. The quotes in the final document are the transcript's own words, located by the model but extracted by our code.

What a "run" means

A run is one theme applied to the whole corpus. At a 55-minute average, that is 1,800 API calls, roughly 24.9 million input tokens and 1.2 million output tokens. At a 150-minute average the same corpus becomes 7,200 calls and 81.2 million input tokens, because each transcript is split into three chunks plus a synthesis call — and the chunk overlap means some text is paid for twice.

> WHERE LONG TRANSCRIPTS HURT
>
> Crossing the 20,000-token threshold turns one API call into four. Long transcripts are disproportionately expensive on *any* provider — which is precisely why an expensive per-token rate compounds rather than merely adds.

The system is already provider-agnostic

This is the crux, and it deserves to be a headline rather than a footnote. The pipeline's client is a standard OpenAI-SDK client pointed at a configurable address. Its configuration file already defaults to:

CURRENT DEFAULTS IN THE PIPELINE CONFIGURATION

SETTING	CURRENT DEFAULT VALUE
Provider	`openrouter`
Endpoint	`https://openrouter.ai/api/v1`
Default model	`deepseek/deepseek-v4-flash`

There is no Claude-specific code to remove. There is no migration to fund. Switching between DeepSeek, Gemini, Claude, or anything else is a change of one configuration value, and the same code path serves all of them. This defuses the largest objection a budget-holder normally has to any technology change — *what will the switch itself cost us?* The answer here is: nothing. It is already done.

We can prove parity rather than assert it

The repository contains an evaluation harness that runs the same transcript and the same theme across several models simultaneously and prints a side-by-side table of relevance score, token usage, and elapsed time. It also records how many passage locations each model found and whether the transcript was chunked.

Underneath that, the pipeline enforces a strict output schema with a retry loop. A model that cannot hold the required JSON format fails loudly and visibly rather than degrading quietly. The code already keeps a list of model endpoints that reject formal structured-output requests and falls back to prompted JSON with strict parse-and-retry — direct

evidence, from our own system, that format compliance genuinely varies between models and that we already detect it.

This turns "trust me, DeepSeek is good enough" into "here is the comparison table, run on our own transcripts." It is the most credible thing in this document, and it is the reason the recommendation ends in a pilot rather than a commitment.

Spending controls that already exist

FISCAL GOVERNANCE BUILT INTO THE PIPELINE

CONTROL	WHAT IT DOES
Global concurrency cap	Total simultaneous API calls are capped (default 12) regardless of how transcript-level and chunk-level parallelism interleave. A run cannot stampede into rate limits or runaway spend.
Resumable checkpointing	Completed transcripts are recorded as they finish. An interrupted run resumes where it stopped instead of re-paying for work already done.
Raw-result cache	Each run's raw analysis payloads are stored, so the Word and HTML reports can be regenerated later — new formatting, new layout, corrected typos — without paying for the analysis again.
Per-request timeout	A 180-second ceiling per call, so one hung request cannot hold a concurrency slot indefinitely.
`--limit N`	Runs the pipeline over the first N transcripts only. This is what makes a costed pilot possible before committing to the full corpus.

One clarification, for accuracy: the cache stores results so reports can be rebuilt for free. It does not deduplicate identical API calls within a run. The saving is real but it is the saving on *re-reporting*, not on re-analysis.

These are worth reading as fiscal governance rather than as features. Spend is bounded, interruptions are not billed twice, and a pilot can be costed before the full commitment is made.

03 The cost analysis

Methodology

A calculator in the repository (`tools/cost_model.py`) derives token volume from the pipeline's own configuration constants rather than from a guess: the 12,000-token chunk size, the 800-token overlap, the 20,000-token single-pass threshold, the 1,655-token analysis system prompt, the 477-token synthesis prompt, and an 8% allowance for schema and rate-limit retries that re-send the full input. It then prices that volume against a table of provider rates.

Anyone can re-run it and get the same answer:

```
python tools/cost_model.py --transcripts 1800 --avg-minutes 55
```

That reproducibility is what makes these numbers credible rather than rhetorical. Every assumption is a named constant that can be challenged individually — they are listed in the appendix.

Volume per full-corpus run — 1,800 transcripts, one theme

MODELLED ESTIMATE · DERIVED FROM PIPELINE CONFIGURATION

AVERAGE TRANSCRIPT	TOKENS EACH	API CALLS	INPUT TOKENS	OUTPUT TOKEN
30 min	~6,100	1,800	15.1 M	1.2 M
55 min (central)	~11,100	1,800	24.9 M	1.2 M
90 min	~18,200	1,800	38.7 M	1.2 M
150 min (chunked)	~30,400	7,200	81.2 M	3.9 M

The jump at the last row is the chunking cliff described in section 2. It is a property of the pipeline, not of any provider.

Cost per full-corpus run, by model

55-MINUTE AVERAGE · PRICES VERIFIED 27 AUG 2026

MODEL	INPUT $/M	OUTPUT $/M	COST PER RUN	PER TRANSCRIPT
Claude Opus 5	$5.00	$25.00	$153.88	$0.0855
Claude Sonnet 5	$2.00	$10.00	$61.55	$0.0342
Claude Haiku 4.5	$1.00	$5.00	$30.78	$0.0171
Gemini 3.6 Flash	$0.75	$3.75	$23.08	$0.0128
Gemini 3.5 Flash Lite	$0.30	$2.50	$10.40	$0.0058
DeepSeek V4 Flash	$0.0679	$0.168	$1.89	$0.0010

150-MINUTE AVERAGE · WORST CASE, TRANSCRIPTS CHUNKED

MODEL	COST PER RUN	PER TRANSCRIPT
Claude Opus 5	$504.16	$0.2801
Claude Sonnet 5	$201.66	$0.1120
Claude Haiku 4.5	$100.83	$0.0560
Gemini 3.6 Flash	$75.62	$0.0420
Gemini 3.5 Flash Lite	$34.18	$0.0190
DeepSeek V4 Flash	$6.17	$0.0034

AN HONEST CORRECTION TO AN EARLIER ESTIMATE

An earlier version of this analysis used $15 / $75 per million tokens for the frontier tier — the previous Opus generation's price. Claude Opus 5 is **$5 / $25**. The frontier tier is now roughly a third of what it was, and the absolute cost of running on it is far less alarming than the earlier figures suggested. We are stating this plainly because the case does not depend on the frightening number. It depends on the ratio, which has actually *widened*: the cheap tier fell further and faster than the expensive one.

The multipliers

COST RATIO ON ONE FULL-CORPUS RUN, 55-MINUTE AVERAGE

COMPARISON	RATIO	WHAT IT MEANS IN PRACTICE
Claude Opus 5 → DeepSeek V4 Flash	81×	The Opus budget for one theme funds 81 themes on the proposed default.
Claude Sonnet 5 → DeepSeek V4 Flash	33×	Even the mid tier costs 33 times the low-cost default.
Claude Opus 5 → Gemini 3.5 Flash Lite	15×	The Western-hosted alternative, if data residency is the deciding concern.
Claude Opus 5 → Gemini 3.6 Flash	6.7×	The most capable of the low-cost options; still under a seventh of the price.

Scenario modelling

FULL PROGRAMME COSTS · 55-MINUTE AVERAGE · MODELLED ESTIMATE

SCENARIO	OPUS 5	SONNET 5	GEMINI 3.5 FLASH LITE	DEEPSEEK V4 FLASH
1 theme × 1,800 transcripts	$153.88	$61.55	$10.40	$1.89
10 themes × 1,800 transcripts	$1,538.82	$615.53	$103.99	$18.90
10 themes + 3 full re-runs (13 passes)	$2,000.47	$800.19	$135.19	$24.57
Hybrid: DeepSeek across all 1,800, plus Claude Sonnet 5 re-scoring the top 150 — per theme			$7.02 ($1.89 bulk + $5.13 QA)	
Hybrid × 10 themes				$70.20
Hybrid × 13 passes				$91.26

The hybrid line is the recommendation, and it is worth pausing on. A full ten-theme research programme with frontier-grade Claude quality assurance on every transcript destined for publication costs **$70** — about **4.6% of the Opus-everything bill**, and 11% of the Sonnet-everything bill. Section 7 explains why that is not a compromise.

Prompt caching: a smaller saving than it sounds

Our analysis system prompt is 1,655 tokens and identical on every call, so it is a natural candidate for prompt caching, which bills repeated prefixes at a fraction of the normal input rate. Both Anthropic and the OpenRouter models support it — Claude at 10% of the

input rate on a cache hit, DeepSeek V4 Flash at $0.0168 per million against $0.0679 standard.

The honest arithmetic: the system prompt is about 12% of the input tokens on a 55-minute transcript. The transcript body — the other 88% — is unique to each call and cannot be cached. Caching the prompt therefore saves roughly **8–9% of a run's total cost**, on any of these models. That is real and worth implementing, but it is a rounding adjustment, not a lever. It does not change any decision in this document. We list it as an implementation to-do, not as a saving to bank in advance.

What OpenRouter itself costs

OpenRouter states that it passes through provider pricing without markup on inference. We verified this directly: Claude Opus 5 lists on OpenRouter at $5 and $25 per million tokens — identical to Anthropic's own published rates. There is no hidden margin on tokens.

OpenRouter monetises instead through a **credit purchase fee of 5.5% (minimum $0.80) on card payments**, and a 5% fee on very high-volume bring-your-own-key usage that does not apply to us. In practice: a $100 approval buys about **$94.50 of usable credit**. We are naming this because a reader should not discover it on a receipt.

---

04 Why the price gap exists

The natural reaction to an 81× price difference is suspicion. If one thing costs 81 times another, surely it is 81 times better, and surely we are about to buy something inferior. That instinct is sound in most markets. It is misleading in this one, and the reason is worth understanding properly.

Capability varies at the frontier. Our task does not live at the frontier.

Models are not equally intelligent. The differences between a frontier model and a mid-tier one are real and, on the right task, dramatic: novel reasoning, long-horizon planning, holding a complex problem in mind across many steps, original synthesis. If we were asking a model to write original theology, or to design a research methodology from scratch, the frontier tier would be worth every cent.

That is not what we are asking. Our pipeline asks a model to do one bounded, well-specified thing, over and over:

- Read a segment of text.
- Judge how relevant it is to a named theme, and how directly.
- Emit strictly-formatted JSON containing that judgement, a short justification, and the verbatim phrases that begin and end each relevant passage.

This is squarely inside what the entire current generation of mid-tier models does reliably. It is careful reading and structured reporting — a commodity operation, performed 1,800 times. At frontier prices, we are paying a specialist's consulting rate for careful transcription.

The capability gap narrowed while the price gap widened

On structured extraction specifically — read this, judge it, report it in this exact shape — the distance between the best model available and the tenth-best has compressed to near-irrelevance, while the price distance has grown. That divergence is the entire opportunity. It is not that the expensive model is bad value in general; it is that it is bad value *for this particular job*.

We are deliberately not quoting benchmark scores here. Published benchmarks measure general capability on tasks that are not ours, and citing them would be borrowing authority we have not earned. The argument does not need benchmarks. It needs the pilot, which measures the only thing that matters: how these models perform on *our* transcripts, against *our* themes, in *our* output format.

The schema is the real quality floor

The pipeline does not trust any model on faith. Every response must validate against a strict schema — score, explicitness rating, reasoning, and location markers with exact verbatim boundaries. A response that does not validate is retried; a model that repeatedly cannot comply fails visibly and gets disqualified. And because the passage markers must be exact verbatim text found in the source, a model that fabricates a quotation produces a marker our extraction code cannot locate. The failure surfaces rather than slipping through into a report.

This architecture is what makes a cheap default safe. The risk of a bad model choice is not silent degradation in a published document; it is a noisy failure at pilot stage that costs a few dollars to discover.

Choosing OpenRouter is not choosing against Claude

Where a task genuinely does need frontier capability, OpenRouter serves Claude Opus 5 on the same key, through the same code path, at the same published price — one configuration line away. Nothing about this proposal reduces our access to Claude, or makes returning to it harder. It makes returning to it a one-line decision instead of a procurement decision.

05 Subscription, API, and our own asset

Three different things tend to get conflated under the single word "Claude." Separating them clarifies what we are actually buying, what we already own, and — most importantly — why the thing we already pay for cannot be pointed at this problem.

SOLD PER TOKEN	**The model** The raw capability — reading, judging, writing. Priced per million tokens. This is what an API purchase buys, and it is the layer where the 81× applies.

SOLD PER SEAT, PER MONTH	**The product wrapper** The Claude chat interface, Projects, file uploads, the desktop and mobile apps. Genuinely excellent, and priced for people rather than for volume. **This is what our subscription buys — and it is a different purchase from the layer above it.** A subscription does not include, and cannot be converted into, access to the model layer for our own software.
BUILT BY US · OWNED BY US	**Our own infrastructure** The chunker, the concurrency gate, the retry-and-schema layer, the checkpointing, the passage extraction, the aggregation, the Word report generator. This is the asset that actually produces the deliverable — and it is provider-neutral by design.

The subscription we already pay for cannot buy this work

This is the point most likely to be misunderstood, and it is worth stating without hedging. **A Claude subscription and Claude API access are two separate products, bought separately, billed separately, on separate accounts.** Paying for one does not give you any amount of the other.

Anthropic states this plainly in its own support documentation:

> "Claude paid plans give subscribers access to Claude on the web, desktop, and mobile... The Claude Console is our developer platform providing API keys and access to Claude models for building applications and integrations. **A paid Claude subscription enhances your chat experience but doesn't include access to the Claude API or Console.**"
>
> Anthropic Help Center, "I have a paid Claude subscription — why do I have to pay separately to use the Claude API and Console?"

The practical consequence for us: our subscription cannot issue an API key. Our pipeline cannot log in with it. There is no setting, no upgrade, and no support request that converts subscription usage into the programmatic access this system needs. The two purchases go through different doors.

TWO SEPARATE PURCHASES, NOT TWO TIERS OF ONE PURCHASE

	CLAUDE SUBSCRIPTION (PRO / MAX / TEAM)	CLAUDE API (CONSOLE)

	CLAUDE SUBSCRIPTION (PRO / MAX / TEAM)	CLAUDE API (CONSOLE)
What you buy	A seat for a person	Tokens for a machine
Priced by	Person, per month	Token, per use
Where it works	Claude on web, desktop and mobile	Any application you write
Gives you an API key	No	Yes — that is what it is
Can run our pipeline	No — not at any subscription tier	Yes
Account & invoice	Separate signup, separate invoice	Separate signup, separate invoice
Unused capacity	Expires monthly	Prepaid credit; does not expire monthly

THE ONE EXCEPTION, STATED HONESTLY

There is a narrow case that a well-informed colleague may raise, and we would rather raise it ourselves. Anthropic does let a Pro or Max subscriber use their plan for some *developer* tools — Claude Code, and projects built on the Claude Agent SDK — through a per-user monthly credit ($20 on Pro, $100 or $200 on the Max tiers). So it is not strictly true that a subscription buys nothing programmatic.

It does not, however, help us, for three independent reasons:

- **Wrong architecture.** That credit applies to Anthropic's own Agent SDK. Our pipeline is a plain OpenAI-compatible HTTP client — deliberately, because that is what makes it provider-neutral. It cannot draw on that credit at all without being rewritten around a single vendor's framework, which is the opposite of what this proposal recommends.
- **Wrong billing unit.** The credit is per user, and Anthropic states it "can't be pooled, transferred, or shared across the organization." A corpus run is organisational work, not one person's work.
- **Explicitly not for this.** Anthropic's own guidance is that "teams running shared production automation should use Claude Platform with an API key for predictable pay-as-you-go billing." We agree with them. That is precisely what this proposal asks for — an API key with pay-as-you-go billing, purchased through OpenRouter so it is not tied to one vendor.

WHAT THIS SETTLES

"We already pay for Claude, so this is duplicate spend" is the objection this section exists to answer, and the answer is that **the two payments cannot substitute for each other in either direction.** The subscription cannot run the pipeline. API credits cannot give the team a chat interface. Declining this request does not save money by falling back on the subscription — it means the corpus does not get analysed at all.

Why the interface could not do this job even if it were free

Set billing aside entirely and the chat interface still cannot do this work. It is superb at what it is for: one person, one conversation, a handful of documents. It is structurally the wrong tool for 1,800 transcripts, and not because of any deficiency in Claude:

WHY THE CHAT INTERFACE CANNOT DO THIS JOB

REQUIREMENT	CHAT INTERFACE	OUR PIPELINE
Processing 1,800 items	A person must be present for every unit of work	Runs unattended, 12 calls in flight
Long transcripts	Upload limits force manual splitting — a person becomes the chunker	Automatic chunking with overlap and synthesis
Interruption	A lost session is lost work	Checkpointed; resumes without re-paying
Output	Prose in a chat window	Validated JSON flowing into a report builder
Costing a piece of work	You can cost a subscription, not a run	Per-run token telemetry; cost per transcript
Reproducibility	Cannot re-run last quarter's analysis and get the same answer	Same inputs, same configuration, same result — and the passage markers show a reviewer exactly how each quote was selected

THE DISTINCTION THAT MATTERS

A subscription seat scales with people. An API key scales with work. Our bottleneck is work, not people. Both purchases are legitimate; they are simply answers to different questions, and only one of them can process a corpus.

06 One key, many applications

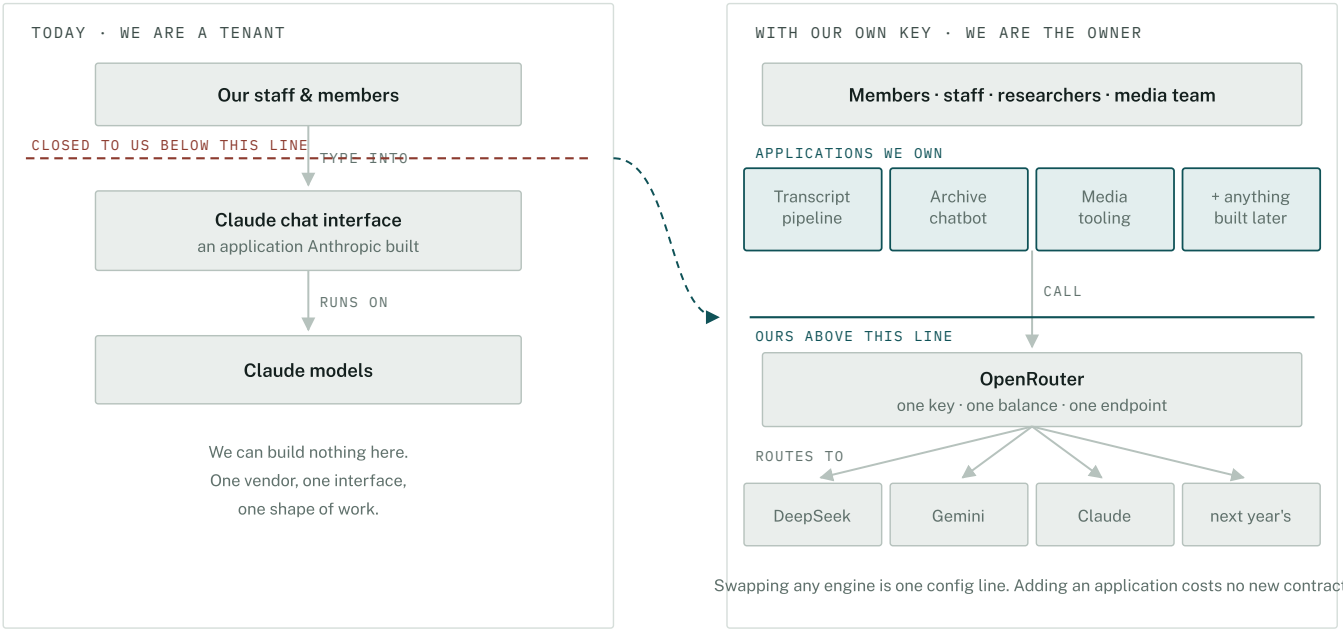
Everything so far has argued about the price of one job. This section argues something larger, and it is the part of the decision with the longest reach: **what we are actually acquiring is not cheaper transcript analysis. It is the ability to build.**

The interface we admire is itself an application

It is worth saying out loud what the Claude chat interface actually is. It is a very good *application* that Anthropic built on top of a model. The conversation window, the memory of what you said earlier, the file uploads, the projects — none of that is the model. It is software wrapped around a model, designed for a general audience of individual users.

Once we hold an API key, we can do the same thing. Not build a competitor to Claude — build the version of it that fits *Gospel Light House specifically*: that knows our archive, speaks to our congregation, follows our workflows, and lives inside the tools our staff already use. The engine is the same class of engine. The difference is that the application around it is ours, shaped to our situation rather than to the average of everyone's.

That is the shift this key represents, and it is a shift in position rather than in price: **from being a tenant in someone else's product to being the owner of our own.**



The same engines. A different boundary. Today the line sits directly beneath Anthropic's application, so the only thing we can do is reach across it and type. With our own key the line drops to beneath *our* applications — the transcript pipeline is simply the first of them — and everything above it is ours to design, change, and own. The dashed arrow is the whole proposal: the boundary moves down.

What could actually be built

These are not hypotheticals from a vendor brochure. Every item below runs on the same key, the same prepaid balance, and the same endpoint the transcript pipeline already

uses — no new procurement, no new vendor, no new approval cycle. They are ordered roughly by how close they are to what we have already built.

THE ARCHIVE AS A RESEARCH CORPUS

Theme extraction pipeline

Scores every transcript against a theme and pulls the exact passages into a Word report. Built, tested, and waiting on a model decision.

BUILT · RESEARCH TEAM

Semantic search across every sermon

Ask "where has the archive dealt with grief in the Psalms?" and get the passages themselves, ranked — not filename matches, not a keyword that happens to appear. Uses embedding models available on the same endpoint.

RESEARCH TEAM · PREACHING STAFF

Scripture citation index

Automatically map every passage of Scripture referenced across all 1,800 sermons, including paraphrases and allusions the text never cites by chapter and verse. Then: "show me everything ever preached on Romans 8."

PREACHING STAFF · ARCHIVISTS

Longitudinal teaching analysis

Track how emphasis on a doctrine or a pastoral concern has changed across years of teaching — the kind of question the corpus can answer but no human has time to ask by hand.

LEADERSHIP · RESEARCH TEAM

SERVING THE CONGREGATION DIRECTLY

Archive question-and-answer assistant

A chatbot that answers from our own teaching and cites back to the specific sermon and moment it drew from. Structurally what the Claude interface does — but grounded in *our* archive and accountable to it, rather than answering from general knowledge.

MEMBERS · PUBLIC

Personalised study plans

A member names a topic or a season they are in; the system assembles a multi-week plan of sermons, passages, and reflection questions drawn from material we have already produced.

MEMBERS · SMALL GROUPS

Small-group discussion guides

Generate a leader's guide from any sermon — summary, discussion questions, relevant passages — as a first draft for a staff member to edit rather than a blank page.

GROUP LEADERS

New-member orientation assistant

Answers the practical questions people are often reluctant to ask a person: what happens in a service, what the church believes about a given matter, how to get involved.

NEWCOMERS

MEDIA, PUBLISHING, AND THE WEEKLY CYCLE

Automatic show notes and summaries

Every recording gets a summary, key points, and passages referenced, generated the moment the transcript exists — work that currently either takes staff hours or does not happen.

MEDIA TEAM

Titles, descriptions, and chapter markers

Draft metadata for YouTube and podcast platforms, including timestamps for each movement of the message. Improves discoverability of material that already exists.

MEDIA TEAM

Clip selection for social

Identify the two or three most self-contained, quotable 60-second segments in a message, with exact start and end points — the same verbatim-boundary technique the transcript pipeline already uses to cut passages.

MEDIA TEAM · COMMUNICATIONS

Devotional and article drafts

Turn one sermon into a five-day devotional or a written article as an editable draft. The theology is already ours; the labour is the reformatting.

COMMUNICATIONS · EDITORS

TRANSLATION AND ACCESSIBILITY

Subtitles and translation

Generate subtitles and translate them into the languages our congregation actually speaks, at a cost per sermon measured in fractions of a cent. This is the item with the widest reach per pound spent.

NON-ENGLISH SPEAKERS · DIASPORA

Plain-language versions

Produce simpler renderings of dense teaching for younger readers, new believers, or anyone reading in a second language — without diluting what was said.

YOUTH · NEW BELIEVERS

Low-bandwidth text digests

Condensed text versions of messages for members who cannot reliably stream an hour of video, distributed by whatever channel actually reaches them.

REMOTE · LOW-CONNECTIVITY MEMBERS

INTERNAL OPERATIONS

Drafting assistance inside our own tools

Announcement drafts, follow-up notes, and correspondence generated where staff already work — in the CRM and admin systems — rather than in a separate chat window they must copy out of.

ADMINISTRATIVE STAFF

Meeting minutes and action extraction

Turn a recorded meeting into minutes with decisions and owners identified. The same transcript machinery, pointed at a different kind of recording.

LEADERSHIP · COMMITTEES

Report and proposal drafting

Assemble first drafts of activity reports and funding applications from material the organisation already holds, for a person to verify and finish.

LEADERSHIP · FUNDRAISING

Different jobs want different engines — which is the argument for a router

Notice something about that list: no single model is the right choice for all of it. A member-facing chatbot needs speed and a low cost per turn because it answers thousands of short questions. Adjudicating a research finding needs the strongest judgement available and runs a few hundred times. Semantic search does not need a chat model at all — it needs an embedding model, a different category of thing entirely.

THE SAME KEY, MATCHED TO THE JOB

JOB	WHAT IT ACTUALLY NEEDS	SENSIBLE ENGINE
Bulk corpus analysis	Very low cost per call at high volume; reliable JSON	DeepSeek V4 Flash
Quality adjudication	Strongest available judgement, low volume	Claude Sonnet 5 or Opus 5

JOB	WHAT IT ACTUALLY NEEDS	SENSIBLE ENGINE
Member-facing chatbot	Fast response, cheap per turn, long context	A Gemini Flash tier
Semantic search	An embedding model — not a chat model	A dedicated embedding model
Translation	Genuine strength in the target languages	Whichever model is strongest in those languages
Staff drafting	Best writing quality; low volume, high visibility	Claude

> WHY THIS SETTLES THE VENDOR QUESTION
>
> No single vendor is the best answer to all six rows, and committing to one means accepting their weakest row as our ceiling. On a router we never have to choose: each application picks the engine that suits it, on the same key and the same balance, and changes its mind later for the cost of one configuration line. **The router is not a cheaper way to buy one model. It is the thing that lets each job have the right one.**

The governance case

This matters more to finance than to engineers. One vendor relationship. One prepaid balance. One dashboard showing spend by application. The alternative — a separate subscription per tool per team — is both more expensive and considerably harder to audit, because each subscription is a fixed monthly charge disconnected from any unit of work.

> FOR THE BUDGET-HOLDER
>
> Prepaid credits mean spend **cannot exceed what was deliberately loaded**. If we load $100, the maximum possible exposure is $100. There is no surprise invoice, no overage, and no auto-renewing commitment. Approving a balance is approving a ceiling, not opening a tap.

What this key does not do

This section has made an expansive claim, so it should end with three honest limits. Naming them is not a retreat from the argument; it is what keeps the argument worth trusting.

LIMITS ON THE "MANY APPLICATIONS" CASE

THE LIMIT	WHAT IT MEANS FOR US
The key removes procurement friction, not build effort.	Every item in the catalogue is a real project needing design, testing, and someone to maintain it afterwards. What the key changes is that none of them requires a new vendor, a new contract, a new invoice, or a new approval — so the question becomes "is this worth building?" rather than "can we get permission to try?" That is a genuine change, but it is not the same as the applications being free.
Public sermon content is low-risk. Member data is not.	The whole data-sovereignty answer in section 8 rests on these being public sermons — already published, containing nothing personal or confidential. Anything touching member records, pastoral notes, giving history, or safeguarding matters is a different risk category and needs its own decision, its own provider restrictions, and probably its own legal review. **This approval should not be read as covering that**, and we are saying so deliberately rather than letting the scope drift later.
Member-facing tools need a higher bar than internal ones.	An assistant answering a member's question about what the church teaches can be confidently wrong, and the cost of that is pastoral, not financial. Anything member-facing needs to cite back to a specific sermon so a person can check it — the same verbatim-passage discipline the transcript pipeline already enforces — and needs human review before launch. The internal and media tools in the catalogue carry no such exposure and could move much sooner.

07 Optionality and hybrid tiering

The durable argument for this decision is not "DeepSeek is cheapest today." Prices and rankings move every few months, and any document that rests on today's leaderboard is obsolete by Christmas. The durable argument is **optionality**.

- **Hundreds of models behind one endpoint and one key.** Including every Claude model, at Anthropic's own prices.
- **Switching is one configuration value** — proven, because it is how the system is already built.
- **Automatic failover.** If a provider has an outage or rate-limits us, the run can continue on another. A single-provider setup simply stops.

- **Price competition works for us continuously.** When something cheaper or better ships, we adopt it the same week rather than renegotiating a contract.
- **No lock-in.** If Claude becomes the right answer next year, we move back — same key, same code, same afternoon.

The hybrid tiering proposal

Here is what we actually recommend running, and it is not a cost-cutting design. It is an allocation design.

TWO-TIER PRODUCTION CONFIGURATION · PER THEME, 55-MINUTE AVERAGE

PASS	MODEL	TRANSCRIPTS	PURPOSE	COST
Bulk	DeepSeek V4 Flash	1,800	Score and locate passages across the whole corpus	$1.89
Adjudication	Claude Sonnet 5	150	Independently re-score the highest-relevance transcripts — the ones destined for publication	$5.13
Total		—	Frontier-grade confidence exactly where a mistake would be visible	$7.02

Note what the second pass buys. It is not a spot check for reassurance. It is a genuine second opinion from a materially different model on every transcript whose findings will actually be quoted in a report — and where the two passes disagree, that disagreement is itself a signal worth reviewing. We get more scrutiny on the published material than a single-model run at any price would give us, because a single model cannot disagree with itself.

> REFRAME
>
> This is not cutting corners. It is **intelligent allocation**: cheap capacity where the work is bulk and verifiable, expensive capacity where the stakes are visible. Spending $154 to apply frontier reasoning uniformly — including to the 1,650 transcripts that turn out to score near zero — is not rigour. It is undirected spending.

08 Risks and objections

A proposal that hides its trade-offs looks naive. Here are the objections we expect, stated as fairly as we can, with our answers.

OBJECTIONS AND RESPONSES

OBJECTION	RESPONSE
"Cheap models will produce worse analysis."	Possibly. We do not have to speculate — the evaluation harness measures it on our own transcripts, and the pilot is designed to answer exactly this. Decide on the evidence, not on this document. The hybrid adjudication pass in section 7 is the standing safeguard if the answer is "slightly worse, but only on hard cases."
"DeepSeek is a Chinese model. What about data sovereignty?"	A fair question, and we will not dodge it. Two things. First, the data: these are sermon transcripts — public teaching, already published in audio form. No personal, financial, or pastoral-confidence data passes through the pipeline. Second, the control: OpenRouter publishes each provider's data-retention and training policy, lets an account opt out of routing to providers that train on submitted data, and allows individual requests to be restricted by data policy; enterprise accounts can additionally route through EU or US regional endpoints. If the concern persists after that, **Gemini 3.5 Flash Lite is a Western-hosted alternative at $10.40 per run** — still 15× cheaper than Opus. That this option exists on the same key, for the price of one config line, *is* the argument for OpenRouter.
"Another vendor to manage."	It replaces per-tool subscriptions with one prepaid account and one dashboard. Fewer relationships, not more — and a spending ceiling instead of a recurring charge.
"What if OpenRouter goes down, or shuts down?"	OpenRouter speaks the standard OpenAI-compatible protocol. Redirecting the pipeline to a provider's own endpoint — Anthropic's, Google's, anyone's — is a change to the endpoint address. The abstraction is the protection. This is the same property that makes the migration free in the first direction.

OBJECTION	RESPONSE
"We already pay for Claude. This is duplicate spend."	It cannot be duplicate spend, because the two purchases are not interchangeable. A Claude subscription "doesn't include access to the Claude API or Console" — Anthropic's own words. It buys seats on the chat apps; it cannot issue an API key, and no subscription tier can run this pipeline. Declining this request would not save money by falling back on what we already pay for; it would mean the corpus is not analysed. Separately, and even setting billing aside, this is batch machine work a chat interface structurally cannot do — no programmatic loop, no checkpointing, no validated output, no per-run costing. See section 5, which also covers the one narrow developer credit that exists on Pro and Max plans and the three reasons it does not apply here.
"Exotic models will be unreliable, or rate-limit us."	Some will. Schema validation with a retry loop catches malformed output; the global concurrency cap of 12 and per-request 180-second timeout bound the blast radius; checkpointing means a failed run resumes rather than restarts. Failure is visible and resumable rather than silent. What we cannot promise in advance is **latency** — cheaper models are not automatically faster, and a slower model on a rate-limited endpoint could make a full run take materially longer. The pilot measures elapsed time per call, which is precisely why it reports latency alongside cost.
"JSON compliance varies by model."	True — and this, not raw intelligence, is the real selection criterion. Our code already carries a fallback path for model endpoints that reject formal structured-output requests, which is direct evidence that the variation is real. The pilot measures it. Any model that cannot hold the format is disqualified at pilot stage for a few dollars, before it is anywhere near a published report.

OBJECTION	RESPONSE
"The savings look too good. What is the catch?"	Three, named plainly. **One:** the corpus average is unmeasured, and if transcripts average 150 minutes rather than 55, every figure here roughly triples. **Two:** the 81× is a ratio between two small numbers — the absolute saving on a single run is $152, which is real but not transformative on its own; the case rests on repetition across themes and iterations. **Three:** we have not yet demonstrated quality parity on our own data. That is what the pilot is for, and it is why this document asks for a pilot rather than a decision.

09 The recommendation

1

Approve a $100 prepaid OpenRouter balance

A prepaid ceiling, not a subscription. After the 5.5% card fee this yields about $94.50 of usable credit — enough for the pilot, a full ten-theme first-year programme under the hybrid configuration, and comfortable margin. Nothing auto-renews; nothing can be spent that was not loaded.

`$100 · one-off · ceiling equals exposure`

2

Approve a paid pilot on 50 transcripts

Run the model comparison harness plus a 50-transcript sample across DeepSeek V4 Flash, Gemini 3.5 Flash Lite, and Claude Sonnet 5 as the control. Report back cost, latency, schema-compliance rate, and score agreement between models. Measure the true corpus average at the same time.

`Estimated $3–5 · approximately one week`

3

On satisfactory results, adopt the two-tier production default

Cheap model across the full corpus; Claude Sonnet 5 re-scoring the top 150 by relevance as adjudication. Review disagreements between the two passes as part of the research workflow, not as an exception.

`~$7 per theme · ~$70 for a ten-theme programme`

4

Retain existing Claude access unchanged

This proposal does not touch the subscription. Interactive and exploratory work stays where it is best served. Nothing here is a reduction in what the team can use.

`No change · no cancellation proposed`

Pilot plan

PROPOSED SEQUENCE, APPROXIMATELY ONE WEEK

STEP	ACTION	OUTPUT
Day 1	Load credit; measure the real corpus — transcript count and true average length	Re-run of the cost model with verified corpus figures, replacing every estimate in section 3

STEP	ACTION	OUTPUT
Days 2–3	Run the comparison harness on a handful of representative transcripts across all candidate models	Side-by-side scores, token usage, latency; models failing the schema are eliminated here
Days 3–4	Run a 50-transcript sample on the surviving candidates plus Claude Sonnet 5 as control	Score agreement between tiers; disagreement cases pulled for human reading
Day 5	Human review of the passages each model selected on the disagreement cases	A judgement on whether the cheap tier's selections are usable
Report	Written recommendation with actual measured cost, latency and agreement rates	A go / no-go on the production default, decided on evidence

ONE CONDITIONAL CAVEAT ON THE FIGURE

If the corpus turns out to average around 150 minutes rather than 55, the hybrid programme costs approximately **$230 for ten themes** rather than $70, and $100 would fund roughly four themes instead of ten. We would come back with a revised figure rather than quietly overspend — but the honest position today is that **$100 is sized against an unverified assumption**, and step one of the pilot exists to replace it.

10 Appendix: sources & assumptions

A. MODEL REFERENCE

VERIFIED 27 AUGUST 2026

MODEL	IDENTIFIER	INPUT $/M	OUTPUT $/M	CACHE READ $/M	C
Claude Opus 5	`anthropic/claude-opus-5`	$5.00	$25.00	$0.50	
Claude Sonnet 5	`anthropic/claude-sonnet-5`	$2.00	$10.00	$0.20	
Claude Haiku 4.5	`anthropic/claude-haiku-4.5`	$1.00	$5.00	$0.10	

MODEL	IDENTIFIER	INPUT $/M	OUTPUT $/M	CACHE READ $/M	C
Gemini 3.6 Flash	`google/gemini-3.6-flash`	$0.75	$3.75	$0.075	
Gemini 3.5 Flash Lite	`google/gemini-3.5-flash-lite`	$0.30	$2.50	$0.03	
DeepSeek V4 Flash	`deepseek/deepseek-v4-flash`	$0.0679	$0.168	$0.0168	

`deepseek/deepseek-v4-flash` — the identifier already configured as our default — was confirmed to exist and to be currently served. Claude models are listed at Anthropic's own published rates on OpenRouter; there is no pass-through markup.

B. PRICING SOURCES

- platform.claude.com/docs/en/about-claude/pricing — all Claude rates, cache multipliers, batch discount. Accessed 27 Aug 2026.
- openrouter.ai/deepseek/deepseek-v4-flash — accessed 27 Aug 2026.
- openrouter.ai/google/gemini-3.6-flash and .../gemini-3.5-flash-lite — accessed 27 Aug 2026.
- openrouter.ai/anthropic/claude-opus-5 — pass-through pricing confirmation. Accessed 27 Aug 2026.
- openrouter.ai/docs/faq — no inference markup; 5.5% card credit fee. Accessed 27 Aug 2026.
- openrouter.ai/docs/features/privacy-and-logging — provider data policy controls. Accessed 27 Aug 2026.

B2. SOURCES ON THE SUBSCRIPTION / API SEPARATION (SECTION 5)

- Anthropic Help Centre — "I have a paid Claude subscription (Pro, Max, Team, or Enterprise). Why do I have to pay separately to use the Claude API and Console?" Source of the quoted passage. Accessed 27 Aug 2026.
- Anthropic Help Centre — "Use the Claude Agent SDK with your Claude plan." Source for the per-user monthly developer credit, the non-pooling rule, and Anthropic's own recommendation that shared production automation use an API key. Accessed 27 Aug 2026.
- Anthropic Help Centre — "Manage usage credits for paid Claude plans." Confirms subscription and usage-credit purchases are billed and invoiced separately. Accessed 27 Aug 2026.

C. COST MODEL ASSUMPTIONS — EACH MAY BE CHALLENGED INDIVIDUALLY

CONSTANTS USED BY `TOOLS/COST_MODEL.PY`

ASSUMPTION	VALUE	BASIS AND SENSITIVITY
Corpus size	1,800	**Unverified.** Scales every figure linearly.
Average transcript length	55 min	**Unverified — the load-bearing assumption.** 90 min raises costs ~45%; 150 min crosses the chunking threshold and roughly triples them.
Speaking rate	150 wpm	Normal preaching and teaching cadence.
Tokens per word	1.35	Standard ratio for conversational English.
Characters per token	3.5	The pipeline's own heuristic, deliberately conservative — it biases toward smaller chunks so real counts stay under budget.
Analysis system prompt	1,655 tok	Measured from the prompt file at 3.5 chars/token. Identical on every call.
Synthesis system prompt	477 tok	Measured. Used only on chunked transcripts.
Output tokens per analysis call	600	**Estimated** from the observed response shape (score, rating, reasoning, several location markers). Should be re-derived from real usage statistics after the pilot.
Output tokens per synthesis call	220	Estimated, as above.
Retry allowance	8%	Conservative real-world allowance for schema and rate-limit retries, which re-send the full input. Set to 0 to see the floor.
Prompt caching	not applied	All figures assume *no* caching. Implementing it would reduce every figure by roughly 8–9%.

D. WHAT IS MEASURED VERSUS WHAT IS ESTIMATED

- **Measured from the code:** chunk size, overlap, single-pass threshold, both system prompt lengths, concurrency caps, retry limits, request timeout, the configured provider and default model.

- **Sourced and dated:** every per-token price, the OpenRouter fee structure, the data-policy controls.
- **Estimated:** corpus size, average transcript length, output tokens per call, retry rate. All four are inputs the pilot will replace with measurements.
- **Not claimed:** any benchmark score, any quantitative statement of relative model quality, any latency figure. We have not measured these and will not assert them.

Prepared for Gospel Light House leadership, 27 August 2026. All cost figures are modelled estimates produced by `tools/cost_model.py` in the project repository and can be reproduced or re-derived under different assumptions by anyone with access to it. Prices were verified on the date shown and change frequently; re-verify before quoting.